

Layered Components Report

Parker Woolsey

9 May 2026

Site Details

I have completed my [layered components page](#) and updated my [GitHub repository](#).

Reflections

Layer Declaration with Vendor CSS

If I were starting a new project with a third party CSS library, I would organize my layers like this:

```
1@layer vendor, tokens, base, components;
```

The `vendor` layer would contain all third party CSS such as Bootstrap or another framework. I would place it first so my own styles could override it more easily without needing overly specific selectors or `!important`.

The `tokens` layer would contain custom properties like colors, spacing, typography, and border radius values. These tokens should come early because the rest of the stylesheet depends on them through `var()`.

The `base` layer would contain bare element styles such as `body`, `headings`, `links`, `lists`, and default typography. This creates a consistent foundation across the site before components are added.

The `components` layer would come last because it contains the most specific styles for things like `cards`, `buttons`, `forms`, and `navigation`. Since it comes later in the cascade, component styles can naturally override base styles when needed.

:is() vs :where()

:is() vs :where()

Both `:is()` and `:where()` help group selectors together and reduce repetition, but they handle specificity differently. `:is()` keeps the specificity of the most specific selector inside it, while `:where()` always has zero specificity.

For example:

```
1 :is(button, .btn)
```

still carries specificity, while:

```
1 :where(h1, h2, h3)
```

adds none.

In a project where other developers will later override base element styles with component classes, I would choose `:where()`. Since it has zero specificity, it makes the base layer easier to override without needing more complicated selectors. That helps keep the cascade cleaner and prevents specificity wars later in development.

Rethinking Unit 1

Rethinking Unit 1

After working with layered CSS, one thing I would change from my Unit 1 assignment is separating my styles into more focused files and layers instead of keeping everything together. In Unit 1, I mixed base styles, layout styles, and component styles too closely, which made the stylesheet harder to scan and maintain.

Now I would split the project into tokens, base, components, and utilities folders. For example, I would move all reusable card and button styles into separate component files instead of leaving them inside a larger stylesheet. I would also rename some variables to be more purpose based instead of appearance based, such as using `--color-primary` instead of a color name.

This layered structure makes the CSS easier to maintain, easier for teams to work with, and much more scalable as projects grow.